Part 1: Intro to Rooflines

Exercise 1. 8 bit = 1 byte

(a) Read bytes: $BD + DF + BF$

(b) Write bytes: $2BF$

(c) Arithmetic intensity: $\frac{2BDF}{BD+DF+BF} \approx \frac{2BDF}{DF} = 2B$

(d)

$$T_{\text{math}} = 2B$$
$$T_{\text{comm}} = \frac{3.94 \times 10^{14}}{2.2 \times 10^{12}} \approx 180$$

Say $B = 240$, then $T_{\text{math}} = 480$. Therefore:

$$T_{\text{lower}} = \max(480, 480) = 480$$
$$T_{\text{upper}} = 480 + 480 = 960.$$

Exercise 2. $\text{bf16}[B, D] \times \text{int8}[D, F] \to \text{bf16}[B, F]$

Read bytes: $2BD + DF$ — Write bytes: $2BF$

Arithmetic intensity:

$$\frac{2BDF}{2BD + DF + 2BF} \approx \frac{2BDF}{DF} = 2B$$

To be compute bound, $2B > 240$, which gives $B > 120$.
So quantizing weights, or decreasing FLOPs, decreases the required batch size to reach the compute-bound regime.

Exercise 3. For $F = D = 4096$:

$$\frac{2B(4096)^2}{4B(4096) + (4096)^2} = \frac{2B(4096)}{4B + 4096}$$

Compute bound condition: $\frac{2B(4096)}{4B + 4096} > 240 \implies B > 135.93.$

For $F = D = 1024$:

$$\frac{2B(1024)^2}{4B(1024) + (1024)^2} = \frac{2B(1024)}{4B + 1024} > 240 \implies B > 225.88.$$

Exercise 4. Load bytes: $BD + BDF$ — Write bytes: BF — FLOPs: $BF \cdot 2D = 2BDF$

Arithmetic intensity:

$$T_{\text{math}} = \frac{2BDF}{BD + BDF + BF} \approx \frac{2BDF}{BDF} = 2.$$

Exercise 5. H100-SXM

Peak arithmetic intensity:

$$T_{\text{comm}} = \frac{\text{Peak FLOPs}}{\text{Bandwidth}} = \frac{(1979/2) \times 10^{12}}{3.35 \times 10^{12}} = \frac{1979}{6.7} \approx 295.$$

Since $\frac{2BDF}{2BD+2DF+2BF} \approx B$, the compute-bound condition is:

$$B > T_{\text{comm}} \implies B > 295.$$

Part 2: How To Think About TPUs

Exercise 1.

$$200 \cdot 2 = 400$$
$$400/32 = 100/8 = 50/4 = 12.5$$

Since VMEM to MXU is very quick, we can disregard its calculation.

$$12.5 \cdot 10^9 / 1.2 \cdot 10^{12} = 12.5/1.2 \cdot 10^3 = 0.01042 = 10.42 \text{ ms}$$

Exercise 2. **TPUv5e**

- CPU Hosts: $256/8 = 32$
- TPU Tensor Cores: 256
- Total FLOPs: $256 \cdot 1.97 \cdot 10^{14}$
- Total HBM: $256 \cdot 16 = 4096$ GB

TPUv5p

- CPU Hosts: $16 \cdot 20 \cdot 28/4 = 2240$

- TPU Tensor Cores: 8960
- Total FLOPs: $8960 \cdot 4.59 \cdot 10^{14}$
- Total HBM: $8960 \cdot 95$ GB

Exercise 3.

$$\frac{2BDF}{2BD + 2DF + 2BF} = \frac{8BD^2}{10BD + 8D^2} \approx \frac{8BD^2}{8D^2} = B$$

$$T_{\text{comms}} = \frac{9.20 \cdot 10^{14}}{1.6 \cdot 10^{10}} = \frac{9.2}{1.6} \cdot 10^4 = 57500$$
$$B > 57500$$

Exercise 4. Bytes to load: $16384 \cdot 4096 + 4096 \cdot B = 67108864 + 4096B$
Bytes to write: $16384 \cdot B$

$$T_{\text{comms}} = (6.7 \cdot 10^7 + 2 \cdot 10^4 \cdot B)/8.2 \cdot 10^{11}$$
$$T_{\text{math}} = (2 \cdot 16384 \cdot 4096 \cdot B)/3.94 \cdot 10^{14}$$
$$T_{\text{math}} > T_{\text{comms}} \implies B > 267$$

This solution assumes best case overlap.

2. VMEM BW is 21-22 times HBM.
T_{comms} becomes 22 times smaller, giving us $B > 12$.

Exercise 5. $2 \cdot 8 \cdot 128 \cdot 8192$ bytes to transfer
6 hops
2 ports to send and recieve data
First byte will arrive in $1/4.9 \cdot 10^{10}$ seconds $+ 6\mu$s

$$= \frac{1}{4.9 \cdot 10^{10}} + \frac{6}{10^6} = 6\mu\text{s}$$

because hopping takes a long time and the other parts are marginal.

Total transfer time:

$$16777216/9 \cdot 10^{10} = 188\mu\text{s} + 6\mu\text{s} = 194\mu\text{s}$$

Exercise 6. 2 ports / host

$$(128 \cdot 1024)^2 = 17179869184 = 16 \text{ GB}$$

16 GB/2 = 8 GB per host

$$\begin{aligned}
\text{PCI} &= 16 \text{ GB over 16 PCIe, so } 1/16 \text{ s} = 63 \text{ ms} \\
\text{ICI} &= 16 \text{ GB}/9 \cdot 10^9 = 167 \text{ ms} \quad \text{(Include hopping latency?)} \\
\text{HBM} \to \text{VMEM} \to \text{MXU} &= 16 \text{ GB}/8.2 \cdot 10^{11} = 20 \text{ ms} \\
\text{FLOPs} &= 2 \cdot 8 \cdot 128 \cdot 1024 \cdot 128 \cdot 1024 = 2.7 \cdot 10^{11}/1.97 \cdot 10^{14} = 1.4 \text{ ms}
\end{aligned}$$

167 ms is the total time assuming pipelining.

Part 3: Sharded Matrices and How to Multiply Them

Exercise 1. Sharded along X, and replicated along Y and Z

$$4 \cdot 2 \cdot 2 = 16 \text{ arrays} \quad \Longrightarrow \quad \frac{1}{16}$$

Exercise 2.

$$\begin{aligned}
\text{AllGather}_x([B_x, D_y]) &= \frac{2 \cdot 1024 \cdot 4096}{4 \cdot 9.0 \cdot 10^{10}} = 23\mu\text{s} \\
\text{AllGather}_{x,y}([B_x, D_y]) &= \frac{1024 \cdot 4096}{9.0 \cdot 10^{10}} = 46\mu\text{s} \quad \text{(Double the bandwidth because there are different physical wires for each axis)} \\
\text{AllReduce}_z([B_y, D_y]\{V_z\}) &= \frac{2 \cdot 2 \cdot 1024 \cdot 4096}{16 \cdot 9 \cdot 10^{10}} = 11.6\mu\text{s} \quad \text{(Each Z shard is 1/16th)}
\end{aligned}$$

Exercise 3.

$$\frac{\frac{256}{4 \cdot 9 \cdot 10^{10}/2} - \frac{64}{9 \cdot 10^{10}/2}}{2} = \frac{0.0000000007}{2} = \frac{0.0007\mu\text{s}}{2}$$

Not explained in the book but we use a unidirectional because of theory overhead, hardware minimums, etc.

Exercise 4. Since our slice is bidirectional, it takes $4/2 = 2\mu$s to AllGather.

1) $\frac{2DF}{W_{ici}} = \text{AllGather}$ — $\frac{2BDF}{C} = T_{\text{math}}$

$$\max\left(\frac{2DF}{W_{ici}}, \frac{2BDF}{C}\right)$$

2) Sharded computation $= \frac{2BDF}{X \cdot C}$ — AllReduce $= \frac{4DF}{W_{ici}}$

$$\max\left(\frac{4DF}{W_{ici}}, \frac{2BDF}{X \cdot C}\right)$$

Strategy 1: $\frac{2BDF}{C} > \frac{2DF}{W_{ici}} \implies B > \frac{C}{W_{ici}}$
Assume $\frac{C}{W_{ici}} = 2550$.
Then we are comms bound when $B < 2550$.
Strategy 2: $\frac{2BDF}{X \cdot C} > \frac{4BF}{W_{ici}} \implies \frac{D}{2X} > \frac{C}{W_{ici}} \implies X < \frac{D}{2(C/W_{ici})}$
$\implies \frac{D}{2X} > 2550 \implies \frac{D}{5100} > X$
Assume $D = 8000$.
$2 > X$ is when Strategy 2 is compute bound.
So both strategies are basically always comms bound.
So which comms is faster?
$\frac{4BF}{W_{ici}} < \frac{2DF}{W_{ici}}$ because $B = 2550$ and $D = 8000$.
Strategy 2 is best when $B < 2550$.

Exercise 5.

1. $A[I_{xyz}, J] \cdot B[J, K]$ then AG
2. $A[I, J] \cdot B[J, K_{xyz}]$ then AG
3. $A[I, J_{xyz}] \cdot B[J_{xyz}, K]$ then AR
4. $A[I, J] \cdot B[J, K]$

1) Assuming bfloat16, total bytes $= 2IJ + 2JK + 2IK$
Bytes per device $= \frac{2IJ+2IK}{64} + 2JK$

$$T_{\text{comms}} \text{ for the matmul} = \frac{2IJ + 2IK}{64 \cdot 1.2 \cdot 10^{12}} + \frac{2JK}{2 \cdot 10^{12}}$$
$$T_{\text{math}} \text{ for the matmul} = \frac{2IJK}{64 \cdot 2.75 \cdot 10^{14}}$$

Now we need to AG $C([I_{xyz}, K])$

$$T_{\text{comms}} \text{ for AG} = \frac{2IK}{64 \cdot 9.0 \cdot 10^{10}}$$
$$\text{Total time} = \max\left(\frac{2IJK}{64 \cdot 2.75 \cdot 10^{14}}, \frac{2IJ + 2IK}{64 \cdot 1.2 \cdot 10^{12}} + \frac{2JK}{1.2 \cdot 10^{12}}\right) + \frac{2IK}{64 \cdot 9.0 \cdot 10^{10}}$$

2) Should be the same as 1), because the operation is the same, just with K sharded instead.

3) $A[I, J_{xyz}] \cdot B[J_{xyz}, K]$
Total FLOPs = $\frac{2IJK}{64}$
Total Bytes = $2IJ + 2JK + 2IK$
Bytes per device = $\frac{2IJ+2JK}{64} + 2IK$

$$T_{\text{comms}} \text{ for matmul} = \frac{2IJ + 2JK + 128IK}{64 \cdot 1.2 \cdot 10^{12}}$$
$$T_{\text{math}} \text{ for matmul} = \frac{2IJK}{64 \cdot 2.75 \cdot 10^{14}}$$

Now we need to AR $C([I, K]\{U_{xyz}\})$
AG = $\frac{2IK}{64 \cdot 9.0 \cdot 10^{10}}$ which we calculated earlier.
So, AR = $\frac{4IK}{64 \cdot 9.0 \cdot 10^{10}}$.

$$\text{Total time} = \max\left(\frac{2IJK}{64 \cdot 2.75 \cdot 10^{14}}, \frac{2IJ + 2JK + 128IK}{64 \cdot 1.2 \cdot 10^{12}}\right) + \frac{4IK}{64 \cdot 9.0 \cdot 10^{10}}$$

4) $A[I, J] \cdot B[J, K]$
Total FLOPs = $2IJK$
Total Bytes = $2IJ + 2JK + 2IK$

$$T_{\text{math}} = \frac{2IJK}{2.75 \cdot 10^{14}}$$
$$T_{\text{comms}} = \frac{2IJ + 2JK + 2IK}{1.2 \cdot 10^{12}}$$

Total Time = $\max(T_{\text{math}}, T_{\text{comms}})$
4) seems too slow given that its denominator has no dimensions to reduce time by.

Looking at 1) and 3), their T_{math} is the same.
Comparing T_{comms} and AG to AR:

1)	**3)**
$\frac{2IJ+2IK+128JK}{64 \cdot 1.2 \cdot 10^{12}}$	$\frac{2IJ+2JK+128IK}{64 \cdot 1.2 \cdot 10^{12}}$
$\frac{2IK}{64 \cdot 9.0 \cdot 10^{10}}$	$\frac{4IK}{64 \cdot 9.0 \cdot 10^{10}}$

Clearly AG is 2 times faster than AR, but T_{comms} for 3) is likely faster than for 1).

This is because J, the contracting dimension, is usually much larger than I.

$$
\begin{aligned}
&I + 128J \quad \text{compared to} \quad J + 128I \\
&I + 128J > J + 128I \\
&\quad 127J > 127I \\
&\quad\quad J > I \quad \implies \quad I < J
\end{aligned}
$$

When $I < J$, which is often, 3) is faster than 1).

Exercise 6. $A[I_x, J_y] \cdot B[J_y, K] \to C[I_x, K]$
We need to AR after matmul.

$$
\begin{aligned}
\text{Total Bytes} &= 2IJ + 2JK + 2IK \\
\text{Bytes per device} &= \frac{2IJ}{16} + \frac{2JK}{4} + 2IK \\
T_{\text{comms}} \text{ for matmul} &= \frac{2IJ + 8JK + 32IK}{16 \cdot 8.2 \cdot 10^{11}} \\
\text{Total FLOPs} &= 2IJK \\
\text{FLOPs per device} &= \frac{2IJK}{16} \\
T_{\text{math}} \text{ for matmul} &= \frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}} \\
\text{Total Time for matmul} &= \max\left(\frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}, \frac{2IJ + 8JK + 32IK}{16 \cdot 8.2 \cdot 10^{11}}\right)
\end{aligned}
$$

Now we need to AR $C([I_x, K]\{U_y\})$.

$$
\text{AR} = 2\text{AG} = 2 \cdot \frac{2IK}{1 \cdot 9.0 \cdot 10^{10}} = \frac{4IK}{9.0 \cdot 10^{10}}
$$

So comms time is $\frac{4IK}{9.0 \cdot 10^{10}}$, and computation time is:

$$
\max\left(\frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}, \frac{2IJ + 8JK + 32IK}{16 \cdot 8.2 \cdot 10^{11}}\right)
$$

$A[I_x, J] \cdot B[J_x, K_y] \to C[I_x, K_y]$
We need to AG on $B[J_x, K_y]$, over x.

$$
T_{\text{comms}} = \frac{2JK}{9.0 \cdot 10^{10}}
$$

Now we have $A[I_x, J] \cdot B[J, K_y] \to C[I_x, K_y]$

$$\text{FLOPs per device} = \frac{2IJK}{16}$$
$$T_{\text{math}} \text{ for matmul} = \frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}$$
$$\text{Bytes per device} = \frac{2IJ}{4} + \frac{2JK}{4} + \frac{2IK}{16}$$
$$T_{\text{comms}} = \frac{8IJ + 8JK + 2IK}{16 \cdot 8.2 \cdot 10^{11}}$$
$$\text{Communication time is } \frac{2JK}{9.0 \cdot 10^{10}}$$
$$\text{Computation time is } \max\left(\frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}, \frac{8IJ + 8JK + 2IK}{16 \cdot 8.2 \cdot 10^{11}}\right)$$

$A[I_x, J] \cdot B[J, K_y] \to C[I_x, K_y]$
We don't need any of the communication primitives.
The time will be based solely on the matmul.

$$T_{\text{comms}} = \frac{\frac{2IJ}{4} + \frac{2JK}{4} + \frac{2IK}{16}}{8.2 \cdot 10^{11}} = \frac{8IJ + 8JK + 2IK}{16 \cdot 8.2 \cdot 10^{11}}$$
$$T_{\text{math}} = \frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}$$
$$\text{Total Time} = \max\left(\frac{2IJK}{16 \cdot 1.97 \cdot 10^{14}}, \frac{8IJ + 8JK + 2IK}{16 \cdot 8.2 \cdot 10^{11}}\right)$$

Exercise 7. 300MB = 300,000,000 bytes.
Since F is the largest size, let's shard it with X and Y.

$$In[B, D] \cdot Win[D, F_{xy}] \cdot Wout[F_{xy}, D] \to Out[B, D]\{U_{xy}\}$$

We can't AG because of the 300MB limit, but we can RS. This would make the output $Out[B_{xy}, D]$.
Since the output needs to match the input sharding, this means In needs to be $In[B_{xy}, D]$.
However, now we have non contracting dimensions sharded along the same axes, so we need to AG on In.

Full pipeline:

$$\text{AG}(In[B_{xy}, D]) \cdot Win[D, F_{xy}] \cdot Wout[F_{xy}, D] \to Out[B, D]\{U_{xy}\} \to \text{RS}(Out) \to Out[B_{xy}, D]$$

Time calculation:

$$\text{AG}(In[B_{xy}, D]) = \frac{2BD}{2 \cdot 9 \cdot 10^{10}} = \frac{128 \cdot 8192}{9 \cdot 10^{10}} = 12\mu\text{s}$$

$In[B, D] \cdot Win[D, F_{xy}]$

$$\begin{aligned} T_{\text{math}} &= \frac{2BDF}{4 \cdot 1.97 \cdot 10^{14}} = \frac{128 \cdot 8192 \cdot 32768}{2 \cdot 1.97 \cdot 10^{14}} = 9\mu\text{s} \\ T_{\text{comms}} &= \frac{\frac{2BD}{4} + \frac{2DF}{4} + \frac{2BF}{4}}{8.2 \cdot 10^{11}} = \frac{8BD + 2DF + 2BF}{4 \cdot 8.2 \cdot 10^{11}} \\ &= \frac{(8 \cdot 128 \cdot 8192) + (2 \cdot 8192 \cdot 32768) + (2 \cdot 128 \cdot 32768)}{4 \cdot 8.2 \cdot 10^{11}} \\ &= 17\mu\text{s} \end{aligned}$$

We are comms bound for the first matmul.

$[B, F_{xy}] \cdot Wout[F_{xy}, D]$

$$\begin{aligned} T_{\text{math}} &= 9\mu\text{s} \quad \text{(same as before)} \\ T_{\text{comms}} &= \frac{\frac{2BF}{4} + \frac{2FD}{4} + 2BD}{8.2 \cdot 10^{11}} \\ &= \frac{(2 \cdot 128 \cdot 32768) + (2 \cdot 32768 \cdot 8192) + (2 \cdot 128 \cdot 8192)}{4 \cdot 8.2 \cdot 10^{11}} \\ &= 17\mu\text{s} \end{aligned}$$

So both matmuls are comms bound.

Now we need to ReduceScatter $Out[B, D]\{U_{xy}\}$.
Since RS is the same runtime as AG,
$\text{RS}_{xyB}\ Out[B, D]\{U_{xy}\} \to Out[B_{xy}, D]$ takes 12μs.

Summing all the communications and computations we get:

$$12 + 17 + 17 + 12 = 58\mu\text{s}$$

Exercise 8. AlltoAll is the cheapest, but AllReduce is not 2 times more expensive than AllGather/ReduceScatter

Exercise 9. **1:** $A[N, M_1] \cdot B[M_1, K], \ldots, A[N, M_x] \cdot B[M_x, K]$
M is sharded and distributed along our devices.

We can use an outer product to get partial sums and then use AR to replicate across all devices.

$$c_{kj} = \sum_i a_{ki} b_{ij}, \quad \text{where } a_{ki} \text{ is a column, and } b_{ij} \text{ is a row.}$$

i values are sharded from the contracting dimension.
Now we have $C[N, K]\{U_x\}$.
We AR to end with $C[N, K]$.

2: Instead of AR, we can RS, which will be much cheaper.

3:

$$\text{AG runtime} = \frac{2NM}{W_{ici}}$$
$$\text{RS runtime} = \frac{2NK}{W_{ici}}$$

So if $K > M$, then AG then matmul is faster than matmul then RS.

Exercise 10. **1:** Total bytes $= N^2$
Bytes per device $= \frac{N^2}{D}$

Ring topology
AG needs $D - 1$ hops to reach every device, so

$$\frac{N^2(D-1)}{D} \text{ bytes in int8 travel along a single ICI link}$$

Note: This is for a single device.

2: You transport less and less data as ATA finishes.
Specifically, we transport $\frac{D(D-1)}{2}$ chunks, because

$$\frac{D(D-1)}{2} = (D-1, \ldots, 1)$$

We assume the matrices to be transported are square, so $\frac{N^2}{D^2}$.
So a single device transports $\frac{N^2(D-1)}{2D}$ bytes in int8.

3: $\frac{AG}{ATA} = \frac{1}{2}$
It is half because we transport less bytes as we go.
The average number of bytes that ATA transports per hop is half that of AG since AG transports the full matrix for every hop, whereas ATA removes an index each hop.

4: Now we can transport the matrices in 2 directions, doubling our speed. Amount of bytes transported in each direction is still $\frac{N^2}{D}$.
Therefore our speed is doubled.

5: We can transport in 2 directions, and also send half the matrix one way, and the other half the other way.
Therefore our speed is quadrupled!

6: $\frac{AG}{ATA} = \frac{1}{4}$, because both are doubled in speed by the added direction, but ATA can also halve the number of bytes sent on an ICI link.

Part 4: Transformer Math

Exercise 1. $3DF + 2D(N+K)H + 2D \ldots$ per layer $+ DV$
$D = 4096,\ F = 16384,\ V = 32000,\ L = 64$

$$3(4096 \cdot 16384) = 201326592$$
$$2 \cdot 4096(4096 + 4096) = 67108864$$
$$64 \cdot (\text{per layer params}) = 64 \cdot 268435456 = 17179869184 \approx 17 \text{ GB}$$

17 GB + 131072000 is still 17 GB

$$\frac{67108864 \cdot 64}{17 \text{ GB}} = \frac{4.3 \text{ GB}}{17 \text{ GB}} = \frac{1}{4} \text{ params are attention}$$

Exercise 2. $2SLKH$ for overall KV cache, so per token is $2LKH$
$2 \cdot 64 \cdot 4096 = 524288$ params

$$2 \cdot 2BDF \text{ FLOPs} \cdot 4 = 8BDF$$
$$\text{Each TPU does } \frac{2BDF}{32} = \frac{BDF}{16} \text{ FLOPs}$$

Exercise 3. I and J contract, so $2IJKLMNO$ FLOPs

Exercise 4.

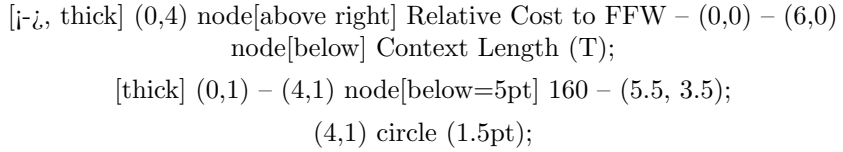
$$\frac{12BTSNH}{2BTNH + 2BSNH} = \frac{6TS}{T+S}$$

$$\text{Arithmetic intensity} = \frac{6TS}{T+S} > \frac{3.94 \cdot 10^{14}}{8.2 \cdot 10^{11}}$$

Assuming MHA,

$$\frac{6T^2 - 3T}{2T} > \frac{3.94 \cdot 10^3}{8.2} \implies T > \frac{3.94 \cdot 10^3}{24.6}$$
$$T > 160$$

Compute bound context is 161.
Note: This is all assuming a training setting and int8 FLOPs.



Exercise 5.

$$12BT^2NH = 12BTD(N+K)H$$
$$TN = D(N+K)$$

Assuming MHA,

$$TN = 2ND$$
$$T = 2D$$

Sequence length must be twice the embedding dimension.

Exercise 6. Softmax$(Q \cdot K^T)V$, then norm(), gelu(), norm().
$4BTSNH$ is the extra flops, the other operations are marginal.

Exercise 7.

$$\frac{6.37\text{B} \cdot 14.8\text{T}}{2.79\text{M} \cdot 1513\text{T}} = \frac{6.37 \cdot 14.8(10^{21})}{2.79 \cdot 1513(10^{18}) \cdot 60} \quad \text{(Amount for second)}$$
$$= \frac{6.37 \cdot 14.8(10^3)}{2.79 \cdot 1513 \cdot 60} = 0.22$$

So 22%

Exercise 8. Assume training setting per token and int8 FLOPs

$$\text{FLOPs} = k \cdot 18(BDF) + 12BTD(N+K)H + 12BT^2NH + 6BTOV$$
$$\text{Bytes} = E(3DF) + 2BT(N+K)H + 2BT^2NH + 2BTOV$$
$$AI = \frac{6Bk}{E}$$

$$\frac{6Bk}{E} > \frac{3.94 \cdot 10^{14}}{8.2 \cdot 10^{11}} \approx 480$$
$$\frac{6Bk}{E} > 480$$
$$\frac{Bk}{E} > 80$$
$$B > \frac{80E}{k}$$

For $E = 256$ and $k = 8$,
$B > 2562.56$ so B must be at least 2560 to be compute bound.

Part 5: How to parallelize a Transformer for training

Exercise 1. $(4DNH + 3DF + D + 2DF)L + DV$

$$= (4(5120)(40)(128) + 3(5120)(13824) + 5120 + 2(5120)(13824)) \cdot 40 + 2(5120)(32000)$$
$$= 18677964800 = 19\text{GB}$$

Note: This includes layer norm and ein sum gates.

Exercise 2. Each parameter needs a momentum and variance state.
$(2 + 4 + 4) \cdot 19\text{GB} = 190\text{GB}$ for params + optimizer states.

For a single token, the checkpointing yields:
$L(2BF + BD)$

Using LLaMA-2 numbers,

$$40(32\text{M} \cdot 13824 + 16\text{M} \cdot 5120) = 2.1 \cdot 10^{13} = 21\text{TB}$$

Note: Checkpointing assumes int-8.

Exercise 3. Note: Realized the textbook uses 13GB for #1 instead of 19GB.

1. Batch size per device = $3\text{M}/16^3 = 732$
   Model parameters = 130GB
   TPUv5p HBM capacity = 96GB
   $96 < 130$, so pure data parallelism is not usable.
2. So we shard D along XYZ.
   Total Params = 130GB

   $$+40(6\text{M} \cdot 13824 + 3\text{M} \cdot 5120) = 4\text{TB}$$

   $$\text{Params per device} = \frac{4000\text{GB} + 130\text{GB}}{16^3} = 1\text{GB}$$

   So the params safely fit on each device.
   As computed earlier, batch size per device is 732.

   $$732 < 2550/3 \implies 732 < 850$$

   So FSDP is comms bound.
3. If FSDP fits on each device, then FSDP+TP will.

   $$\frac{2550^2}{2 \cdot 13824} = 235$$

   $732 > 235$, so we are compute bound.
   FSDP+TP is usable.

   $$X_{\text{opt}} = \sqrt{\frac{B}{F} \cdot 2 \cdot 16^3} = 1333$$
   $$Y = 3$$

   Everything is sharded, so each device has the same amount of memory used as FSDP, so 1GB.
   Ignoring attention FLOPs,

   $$\frac{6 \cdot 3000000 \cdot 130 \cdot 10^9}{4096 \cdot 0.4 \cdot 4.59 \cdot 10^{14}} = 300\text{ms}$$

Part 6: Training LLaMA3 on TPUs

Exercise 1. FSDP+TP for each slice, and DP across pods.

Per training step:

$$T_{\text{comms}} = \frac{4 \cdot 70\text{B}}{8960 \cdot 6.25\text{GB}} = 5\text{ms}$$
$$T_{\text{math}} = \frac{6 \cdot 70\text{B} \cdot B}{4 \cdot 8960 \cdot 4.59 \cdot 10^{14}} = B(2.6) \cdot 10^{-8}$$

$$\begin{aligned} B(2.6) \cdot 10^{-8} &> 5 \cdot 10^{-3} \\ 2.6B &> 5 \cdot 10^{5} \\ B &> \frac{5}{2.6} \cdot 10^{5} \\ B &> 192\text{K} \end{aligned}$$

As long as the batch size is greater than 200K, we are compute bound.

Therefore, our per step training time is:

$$2.6(200\text{K}) \cdot 10^{-8} = 5.2\text{ms}$$

Exercise 2. a)

hyperparam	**value**
L	126
D	16384
F	53248
N	128
K	8
H	128
V	128256

$$\begin{aligned} \text{Total params} &= 126(3 \cdot 16384 \cdot 53248 + 2 \cdot 16384 \cdot 128 \cdot 128 \\ &\quad + 2 \cdot 16384 \cdot 8 \cdot 128 + 16384) + 2 \cdot 16384 \cdot 128256 \\ &= 405 \text{ Billion} \end{aligned}$$

FLOPs per training step $= 6 \cdot 405\text{B} = 2.43\text{T}$ per token

$$\begin{aligned} \text{FLOPs for 15T tokens} &= 15\text{T} \cdot 2.43\text{T} = 36.45 \cdot 10^{24} \\ &= 36.45 \text{ septillion} \end{aligned}$$

b) Can't use DP because params are too large, need to use Mixed FSDP+TP

$$T_{\text{comms}} = \frac{4 \cdot 405\text{B}}{8960 \cdot 6.25\text{GB}} = 29\text{ms}$$
$$T_{\text{math}} = \frac{6 \cdot 405\text{B} \cdot B}{8 \cdot 8960 \cdot 4.59 \cdot 10^{14}} = B(7.4) \cdot 10^{-8}$$

$$B(7.4) \cdot 10^{-8} > 29 \cdot 10^{-3}$$
$$B > \frac{29}{7.4} \cdot 10^{5}$$
$$B > 392\text{K}$$

Compute bound when B is 400K or higher.
Per step training time = $(400\text{K})(7.4) \cdot 10^{-8} = 29.6$ ms

Part 7: Inference

Exercise 1. $L(3DF + 2D(N+K)H + D) + 2DV$

$$\begin{aligned} &= 64((3 \cdot 4096 \cdot 16384) + 2 \cdot 4096(32+8)256) + 4096) \\ &\quad + 2 \cdot 4096 \cdot 32128 \\ &= 18.5 \text{ billion parameters} \end{aligned}$$

KV cache size = $2SLKH = 2S \cdot 64 \cdot 8 \cdot 256 = S(0.262)$ MB

Exercise 2. KV cache = 128000(0.262)MB = 33.5GB

$$\text{Params per device} = \frac{18.5\text{B}}{16} = 1.16\text{B} = 1.16\text{GB}$$
$$\text{TPUv5e HBM capacity} = 16\text{GB}$$
$$\text{Amount left after params} = 16 - 1.16 = 14.84\text{GB}$$
$$\text{Amount of memory left} = 14.84 \cdot 16 = 237.44\text{GB}$$
$$\text{Max batch size} = \frac{237.44}{33.5} = 7$$

If the KV heads were dropped to 1, then max batch size would be 56.
Note: If $K = 1$, then model's total params changes too.

Exercise 3.

$$\frac{18.5\text{B}}{16 \cdot 8.2 \cdot 10^{11}} = 1.4 \text{ milliseconds}$$

Note: This assumes full HBM bandwidth usage.

Exercise 4.

1. 4 bidirectional links per TPU via ICI wires.
2. $Y > F/(B \cdot \beta)$

$$\begin{aligned} \beta &= \frac{8.2 \cdot 10^{11}}{9.0 \cdot 10^{10}} = \frac{82}{9} = 9.1 \\ Y &> \frac{16384}{(7 \cdot 9.1)} \\ Y &> 257 \end{aligned}$$

   We can do TP up to 257 ways without significantly impacting throughput.

3. We have GQA, so we shard heads and then batches.

$$\frac{8 \cdot 33.5\text{GB} + 18.5\text{B}}{16 \cdot 8.2 \cdot 10^{11}} + \frac{2 \cdot 2 \cdot 4096}{9.0 \cdot 10^{10}} = 22 \text{ milliseconds per step}$$

   Note: GQA only requires an All Reduce.

Exercise 5.

1.

$$\begin{aligned} &64(16 \cdot 3 \cdot 4096 \cdot 16384 + 2 \cdot 4096 \cdot 40 \cdot 256 + 4096) \\ &\quad + 2 \cdot 4096 \cdot 32128 \\ &= 211.8 \text{ billion total params} \end{aligned}$$

$$\begin{aligned} &64(2 \cdot 3 \cdot 4096 \cdot 16384 + 2 \cdot 4096 \cdot 40 \cdot 256 + 4096) \\ &\quad + 2 \cdot 4096 \cdot 32128 \\ &= 31.4 \text{ billion activated params} \end{aligned}$$

2. Whatever the MoE bound was, it will increase by $E/K = 8$.

$$240 \cdot 8 = 1920$$

3. $2S \cdot 64 \cdot 8 \cdot 256 = S(0.262)\text{MB}$
4. $2 \cdot 31.4 \text{ billion} \cdot T = 62.4 \text{ billion flops per token}$

Exercise 6. **1.**

$$\text{Params per device} = \frac{211.8\text{B}}{128} = 1.65\text{B}$$
$$\text{Loading time} = \frac{1.65 \cdot 10^9}{8.2 \cdot 10^{11}} = 2 \text{ milliseconds}$$
$$\text{Free HBM per TPU} = 16 - 1.65 = 14.35$$

2.

$$\frac{211.8 \cdot 10^9}{16 \cdot 10^9} = \frac{211.8}{16} = 14 \text{ devices, so a } 4 \times 4 \text{ slice}$$

Exercise 7.

$$T_{\text{comms}} = \frac{2FD}{X \cdot 2W_{ici}} + \frac{4BF}{YZ \cdot W_{ici}} + \frac{2BD}{X \cdot 2W_{ici}}$$

Assume $F = 4D$, and $X = \sqrt{N/8},\ YZ = \sqrt{8N}$

$$T_{\text{comms}} = \frac{\sqrt{128}BD}{\sqrt{N} \cdot W_{ici}}$$

T_{math} is $4BDF/N \cdot C$

$$\begin{aligned} TP_{\text{comms}} &= 4BD/3 \cdot W_{ici} \\ TP_{\text{comms}} &> T_{\text{comms}} \\ \frac{4BD}{3 \cdot W_{ici}} &> \frac{\sqrt{128}(BD)}{\sqrt{N} \cdot W_{ici}} \\ \frac{4}{3} &> \frac{\sqrt{128}}{\sqrt{N}} \\ \frac{16}{9} &> \frac{128}{N} \\ N &> 72 \end{aligned}$$

So 2D model sharding wins when we have more than 72 chips.

Part 8: Serving LLaMA 3

Exercise 1. FLOPs per token = $2 \cdot 405$ billion = 810 billion
TPUv5e peak bfloat16 FLOPs = $1.97 \cdot 10^{14}$ per second

$$810 \cdot 10^9/(1.97 \cdot 10^{14} \cdot N) =$$

$$\text{FLOPs bound} = \frac{4.1 \text{ milliseconds}}{N}$$
$$\text{Comms bound} = \frac{405 \cdot 10^9}{N \cdot 8.2 \cdot 10^{11}} = \frac{0.5}{N} \text{ seconds}$$

Exercise 2. Model params = 8GB

$$\text{KV cache} = 2SLKH = 2S \cdot 32 \cdot 8 \cdot 128 = 65536(S)$$

Assuming max sequence length $S = 8192$,

$$\text{KV cache} = 0.54\text{GB}$$
$$\text{Total KV caches} = 240 \cdot 0.54\text{GB} = 129\text{GB}$$
$$\text{Peak working activations} = BF = 240 \cdot 14336 = 0.0034\text{GB}$$
$$\text{Total Bytes} = 8\text{GB} + 129\text{GB} + 0.0034\text{GB} = 137\text{GB}$$

$$\frac{137\text{GB}}{16\text{GB}} = 8.5625$$

So the smallest topology we can use is 4×4.

Exercise 3. I think we first want to see what the min topology is. So lets assume batch size = 1.

$$\text{Model Params} = 405\text{GB}$$
$$\text{KV cache size} = 2SLKH = 2 \cdot 131072 \cdot 126 \cdot 8 \cdot 128 = 33.8\text{GB}$$
$$\text{Peak working activations} = BF = F = 53248 = 0.05\text{MB}$$
$$\text{Total bytes} = 438.8\text{GB}$$

$$\frac{438.8\text{GB}}{16\text{GB}} = 27.425$$

So min topology is 4×8.
This min assumes $B = 1$. So 8×8 should be able to process more batches.
Since this is inference, we need to TP to shard activations.

$$15\text{ms} = \frac{B \cdot 33.8\text{GB}}{64 \cdot 8.2 \cdot 10^{11}} + \max\left(\frac{2B \cdot 405\text{B}}{64 \cdot 1.97 \cdot 10^{14}}, \frac{405\text{GB}}{64 \cdot 8.2 \cdot 10^{11}}\right)$$

To maximize throughput we need to maximize batch size. Best case scenario is that T_{math} for the MLP is greater than T_{comms}.

$$15\text{ms} = B(0.64\text{ms}) + B(0.064\text{ms})$$
$$15\text{ms} = B(0.704\text{ms})$$
$$B = 21.31$$

So max batch size under these conditions is 21.
Lets check to see if the MLP is FLOPs bound at $B = 21$.

$$21(0.064\text{ms}) = 1.34\text{ms}$$
$$\frac{405\text{GB}}{64 \cdot 8.2 \cdot 10^{11}} = 7.72\text{ms}$$

We are still mem bandwidth bound. Lets calculate B with the new bound for the MLP.

$$15\text{ms} = B(0.64)\text{ms} + 7.72\text{ms}$$
$$7.28\text{ms} = B(0.64)\text{ms}$$
$$B = 11.37$$

Max batch size is 11.

$$\frac{11}{15\text{ms}} \cdot 1000 = 733 \text{ tokens/s}$$

Highest throughput at 15ms / token is 733 tokens per second.

For min step time, B must be 1. As I computed earlier,

$$\text{Latency} = B(0.64) + 7.72 = 8.36 \text{ ms}$$

Min step time is 8.36 milliseconds.

Part 9

The exercises couldn't be done as Colab no longer supports TPUv2-8 runtimes.

Part 10: JAX Implementation

1.1

```
import numpy as np
import jax
import jax.numpy as jnp
import jax.sharding as shd
jax.config.update('jax_num_cpu_devices', 8)
```

shard_map

```
mesh = jax.make_mesh((4,2), ('x', 'y'), (shd.AxisType.
    Explicit, shd.AxisType.Explicit))
jax.set_mesh(mesh)

x = jnp.arange(0, 512, dtype=jnp.int32).reshape(64,8)
x = jax.device_put(x, jax.NamedSharding(mesh, jax.P('x','y')
    ))

jax.debug.visualize_array_sharding(x)

@jax.shard_map(in_specs=jax.P('x', 'y'), out_specs=jax.P('x'
    ,'y'))
def average(x):
  return x.mean().reshape(1,1)

out = average(x)
out
```

jax.jit

```
def average(x):
  X = mesh.shape['x']
  Y = mesh.shape['y']
  arr = jax.lax.reshape(x,(X, x.shape[0] // X, Y, x.shape[1]
       // Y), out_sharding=shd.NamedSharding(mesh, jax.P('x',
      'y')))
  return arr.mean(axis=(1,3))

x = jnp.arange(0, 512, dtype=jnp.int32).reshape(64,8)

x = jax.device_put(x, jax.NamedSharding(mesh, jax.P('x','y')
    ))

avg = jax.jit(average, out_shardings=jax.P('x','y'))

out1 = avg(x)

np.testing.assert_array_equal(out, out1)
```

1.2

```
mesh = jax.make_mesh((4,2), ('x', 'y'), (shd.AxisType.
    Explicit, shd.AxisType.Explicit))
jax.set_mesh(mesh)

x = jnp.arange(0, 512, dtype=jnp.int32).reshape(64,8)
x = jax.device_put(x, jax.NamedSharding(mesh, jax.P('x','y')
    ))

jax.debug.visualize_array_sharding(x)

@jax.shard_map(in_specs=jax.P('x', 'y'), out_specs=jax.P('x'
    ,'y'))
def roll(x):
  return (jnp.roll(x, 62, axis=0) - x)

out = roll(x)
out
```

2.1

```
def local_moe(W,A,B):
  E,_,_ = W.shape
  S,_ = A.shape
  expert_indices = jnp.arange(E)[:,None]
  mask = B[None,:]==expert_indices # ES

  A_masked = A[None, :,:] * mask[:,:, None] # ESD

  out = A_masked @ W # ESF
  return out.sum(axis=0) #SF

E = 400
S = 800
D = 400
F = 200

W = jnp.zeros((E,D,F))
A = jnp.zeros((S,D))
B = jnp.zeros((S,), dtype=jnp.int32)

jitted_local_moe = jax.jit(local_moe)
```

2.2

```
import time
start_clock = time.perf_counter()

out = jitted_local_moe(W,A,B)
out.block_until_ready()
end_clock = time.perf_counter()

print(f"{(end_clock-start_clock) * 1000}ms")
print(out)
```

2.3.1

```
@jax.shard_map(mesh=mesh, in_specs=((jax.P('x',None,None),
    jax.P('x',None),jax.P('x'))), out_specs=jax.P('x',None))
    #out is S_x, D
def moe_all_gather(W,A,B):
  #all gather the tokens and routings, but keep weights
      sharded
  A_global = jax.lax.all_gather(A, axis_name='x', tiled='
      True')
  B_global = jax.lax.all_gather(B, axis_name='x', tiled='
      True')

  E_local, _, _ = W.shape
  #figure out what experts we have locally
  idx = jax.lax.axis_index('x')
  offset = idx * E_local
  expert_indices = jnp.arange(E_local)[:,None] + offset

  #same masking as before, here we need to mask the full
      data
  mask = B_global[None,:]==expert_indices
  A_masked = A_global[None, :,:] * mask[:,:, None] # ESD

  out = A_masked @ W # ESF
  out = out.sum(axis=0)

  #we have partial sums, so we need to all reduce
  out = jax.lax.psum(out, axis_name='x')

  S_local = A.shape[0]

  return jax.lax.dynamic_slice_in_dim(out,
                                      #start idx
                                      idx*S_local,
                                      #slice length
```

```
                                        S_local, axis=0) #
                                            axis 0 because we
                                            want the tokens (S)
                                             from (S,F)
```

2.3.2

```
@jax.shard_map(mesh=mesh, in_specs=((jax.P('x',None,None),
    jax.P('x',None),jax.P('x'))), out_specs=jax.P('x',None))
    #out is S_x, D
def moe_ragged(W,A,B):
  E_local, D, F = W.shape
  S_local, _ = A.shape
  num_devices = jax.lax.axis_size('x')

  target_devices = B // E_local #global token to device
      indices
  local_expert_idxs = B % E_local

  sort_idxs = jnp.argsort(target_devices)
  unsort_idxs = jnp.argsort(sort_idxs) #same trick as with
      the moe transformer

  A_sorted = A[sort_idxs]
  targets_sorted = target_devices[sort_idxs]
  experts_sorted = local_expert_idxs[sort_idxs]

  send_counts = jnp.bincount(targets_sorted, length=(
      num_devices)) #tokens for each device
  offsets = jnp.cumsum(send_counts) - send_counts

  #compute max amount of iterations
  global_iters = jax.lax.pmax(jnp.max(send_counts),
      axis_name='x') #max tokens across devices

  #stop condition
  def cond_fn(state):
    i, _ = state
    return i < global_iters

  def body_fn(state):
    i, out_buf = state

    fetch_idxs = offsets + i #moving onto next token for
        each device
    mask = i<send_counts #if we've moved more times than
        device has tokens, then output false
```

```
    safe_fetch_idxs = jnp.where(mask, fetch_idxs, S_local) #
        use S_local as the fallback value because it will
        lead to a 0.0 later

    tokens_to_send = A_sorted.at[safe_fetch_idxs].get(mode='
        fill', fill_value = 0.0)
    #Also need to send local experts
    experts_to_send = experts_sorted.at[safe_fetch_idxs].get
        (mode='fill', fill_value = 0)

    tokens_recieved = jax.lax.all_to_all(tokens_to_send,
        axis_name='x', split_axis=0, concat_axis=0)
    experts_recieved = jax.lax.all_to_all(experts_to_send,
        axis_name='x', split_axis=0, concat_axis=0)

    active_W = W[experts_recieved]

    features = jnp.einsum('nd,ndf->nf', tokens_recieved,
        active_W)

    returned_features = jax.lax.all_to_all(features,
        axis_name='x', split_axis=0, concat_axis=0)#send
        tokens back to original devices

    out_buf = out_buf.at[safe_fetch_idxs].set(
        returned_features, mode='drop') #returns back the
        tokens and drops the S_local values

    return (i+1, out_buf)

  out_sorted = jnp.zeros((S_local, F), dtype=A.dtype)
  init_state = (0, jax.lax.pvary(out_sorted, ('x',)))
  _, final_out = jax.lax.while_loop(cond_fn, body_fn,
      init_state)

  return out_sorted[unsort_idxs]
```

```
import time

E = 400
S = 800
D = 400
F = 200

W = jnp.zeros((E,D,F))
A = jnp.zeros((S,D))
B = jnp.zeros((S,), dtype=jnp.int32)

start_clock = time.perf_counter()
```

```
out = moe_all_gather(W,A,B)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")

start_clock = time.perf_counter()

out = moe_ragged(W,A,B)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

2.4

```
@jax.shard_map(mesh=mesh, in_specs=((jax.P('x',None,None),
    jax.P('x',None),jax.P('x'))), out_specs=jax.P('x',None))
    #out is S_x, D
def moe_ragged_topk(W,A,B):
  E_local, D, F = W.shape
  S_local, _ = A.shape
  K = B.shape[1]

  num_devices = jax.lax.axis_size('x')

  A_pairs = jnp.repeat(A,K,axis=0)
  experts_global = B.reshape(-1) # remove D

  target_devices = experts_global // E_local
  local_expert_idxs = experts_global % E_local

  sort_idxs = jnp.argsort(target_devices)
  unsort_idxs = jnp.argsort(sort_idxs)

  A_sorted = A_pairs[sort_idxs]
  targets_sorted = target_devices[sort_idxs]
  experts_sorted = local_expert_idxs[sort_idxs]

  num_pairs = S_local * K

  send_counts = jnp.bincount(targets_sorted, length=
      num_devices)
```

```
offsets = jnp.cumsum(send_counts) - send_counts

global_iters = jax.lax.pmax(jnp.max(send_counts),
    axis_name='x') #max tokens across devices

#stop condition
def cond_fn(state):
  i, _ = state
  return i < global_iters

def body_fn(state):
  i, out_buf = state

  fetch_idxs = offsets + i #moving onto next token for
      each device
  mask = i<send_counts #if we've moved more times than
      device has tokens, then output false
  safe_fetch_idxs = jnp.where(mask, fetch_idxs, num_pairs)
       #use num_pairs as the fallback value because it will
       lead to a 0.0 later

  tokens_to_send = A_sorted.at[safe_fetch_idxs].get(mode='
      fill', fill_value = 0.0)
  #Also need to send local experts
  experts_to_send = experts_sorted.at[safe_fetch_idxs].get
      (mode='fill', fill_value = 0)

  tokens_recieved = jax.lax.all_to_all(tokens_to_send,
      axis_name='x', split_axis=0, concat_axis=0)
  experts_recieved = jax.lax.all_to_all(experts_to_send,
      axis_name='x', split_axis=0, concat_axis=0)

  active_W = W[experts_recieved]

  features = jnp.einsum('nd,ndf->nf', tokens_recieved,
      active_W)

  returned_features = jax.lax.all_to_all(features,
      axis_name='x', split_axis=0, concat_axis=0)#send
      tokens back to original devices

  out_buf = out_buf.at[safe_fetch_idxs].set(
      returned_features, mode='drop') #returns back the
      tokens and drops the S_local values

  return (i+1, out_buf)

out_sorted = jnp.zeros((num_pairs, F), dtype=A.dtype)
init_state = (0, jax.lax.pvary(out_sorted, ('x',)))
```

```
  _, out_sorted = jax.lax.while_loop(cond_fn, body_fn,
      init_state)

  out_pairs = out_sorted[unsort_idxs]

  out = out_pairs.reshape(S_local, K, F)

  return jnp.mean(out, axis=1) #mean over experts (K) out of
      (S,K,F)

E = 400
S = 800
D = 400
F = 200
K = 100
W = jnp.zeros((E,D,F))
A = jnp.zeros((S,D))
B = jnp.zeros((S,K), dtype=jnp.int32)

start_clock = time.perf_counter()

out = moe_ragged_topk(W,A,B)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

3.1

```
@jax.shard_map(mesh=mesh, in_specs=(jax.P('x','y'), jax.P('y
    ', None)), out_specs=jax.P('x', None))
def naive(A_local,W_local):
  out = A_local @ W_local
  out = jax.lax.psum(out, axis_name='y') #all reduce over
      contracting axis 'y'
  return out

import time

B = 800
D = 400
F = 200

W = jnp.zeros((D,F))
A = jnp.zeros((B,D))
```

```
start_clock = time.perf_counter()

out = naive(A,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

```
@jax.shard_map(mesh=mesh, in_specs=(jax.P('x','y'), jax.P('y
    ', None)), out_specs=jax.P('x', None))
def overlapped_allreduce(A,W):
  num_tiles=4
  B_X, D_Y = A.shape
  _, F = W.shape #dont need to extract first dim of W
      because it is D_Y
  assert F % num_tiles == 0
  tile_size = F // num_tiles

  W_tiled = W.reshape (D_Y, num_tiles, tile_size).transpose
      (1,0,2) #so jax.lax.scan will iterate over num_tiles

  def f(carry, W_tile):
    out = A @ W_tile
    out = jax.lax.psum(out, axis_name='y')

    return carry, out

  _, stacked_outs = jax.lax.scan(f=f, init=None, xs=W_tiled)
  final_out = stacked_outs.transpose(1,0,2).reshape(B_X,F) #
      swap num_tiles with B_X, and reconstruct F

  return final_out

start_clock = time.perf_counter()

out = overlapped_allreduce(A,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

3.2

```
@jax.shard_map(mesh=mesh, in_specs=(jax.P('x','y'), jax.P('y
    ', None)), out_specs=jax.P('x','y'))
```

```
def overlapped_reduce_scatter(Tmp, W2):
  axis_size = jax.lax.axis_size('y')
  axis_index = jax.lax.axis_index(axis_name='y')

  B_local, F_local = Tmp.shape
  _, D = W2.shape
  D_local = D // axis_size

  W2_tiles = jnp.reshape(W2, (F_local, axis_size, D_local))

  step_0_chunk = (axis_index + axis_size - 1) % axis_size
  acc = Tmp @ W2_tiles[:, step_0_chunk, :]

  def f(acc, step):
    perm = [(i, (i+1) % axis_size) for i in range(axis_size)
        ] #(0,1) (1,2) (2,3) (3,0)
    next_acc = jax.lax.ppermute(acc, 'y', perm=perm) #shift

    chunk_idx = (axis_index + axis_size - 1 - step) %
        axis_size
    local_val = Tmp @ W2_tiles[:, chunk_idx, :]

    return next_acc+local_val, None

  steps = jnp.arange(1, axis_size)
  final_acc, _ = jax.lax.scan(f, acc, steps)

  return final_acc

import time

B = 800
D = 400
F = 200

W = jnp.zeros((F,D))
Tmp = jnp.zeros((B,F))

start_clock = time.perf_counter()

out = overlapped_reduce_scatter(Tmp,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

3.3

```
@jax.jit
def jitted_transformer_block(In, W_in, W_out):
  h = In @ W_in
  out = h @ W_out
  return out

import time

B = 1024
D = 4096
F = 2048

In = jnp.zeros((B,D))
W_in = jnp.zeros((D,F))
W_out = jnp.zeros((F,D))

start_clock = time.perf_counter()

out = jitted_transformer_block(In,W_in,W_out)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

```
@jax.shard_map(mesh=mesh, in_specs=(jax.P('x','y'), jax.P('y
    ', None), jax.P('y', None)), out_specs=jax.P('x','y'))
def sh_transformer_block(In, W_in, W_out):
  axis_size = jax.lax.axis_size('y')
  axis_index = jax.lax.axis_index(axis_name='y')

  num_tiles=4
  B_X, D_Y = In.shape
  _, F = W_in.shape
  assert F % num_tiles == 0
  tile_size = F // num_tiles

  W_tiled = W_in.reshape(D_Y, num_tiles, tile_size).
      transpose(1,0,2) #so jax.lax.scan will iterate over
      num_tiles

  def all_reduce_step(carry, W_tile):
    out = In @ W_tile
    out = jax.lax.psum(out, axis_name='y')
    return carry, out

  _, stacked_outs = jax.lax.scan(all_reduce_step, None,
```

```
      W_tiled)

  H_full = stacked_outs.transpose(1,0,2).reshape(B_X,F)


  F_local = F // axis_size
  start_idx = axis_index * F_local

  Tmp = jax.lax.dynamic_slice(H_full, (0, start_idx), (B_X,
      F_local))

  _, D = W_out.shape
  D_local = D // axis_size

  W_out_tiles = jnp.reshape(W_out, (F_local, axis_size,
      D_local))

  step_0_chunk = (axis_index + axis_size - 1) % axis_size
  acc = Tmp @ W_out_tiles[:, step_0_chunk, :]

  def f(acc, step):
    perm = [(i, (i+1) % axis_size) for i in range(axis_size)
        ] #(0,1) (1,2) (2,3) (3,0)
    next_acc = jax.lax.ppermute(acc, 'y', perm=perm) #shift

    chunk_idx = (axis_index + axis_size - 1 - step) %
        axis_size
    local_val = Tmp @ W_out_tiles[:, chunk_idx, :]

    return next_acc+local_val, None

  steps = jnp.arange(1, axis_size)
  final_acc, _ = jax.lax.scan(f, acc, steps)

  return final_acc

start_clock = time.perf_counter()

out = sh_transformer_block(In,W_in,W_out)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

Note: This is 1.66 times faster than a jit implementation, with B=1024, D=4096, F=2048.

4.0

```
@jax.shard_map(mesh=mesh, in_specs=(jax.P('x','y'), jax.P('y
    ', None)), out_specs=jax.P('x','y'))
def bidirectional_reduce_scatter(Tmp, W2):
  axis_size = jax.lax.axis_size('y')
  axis_index = jax.lax.axis_index(axis_name='y')

  B_local, F_local = Tmp.shape
  _, D = W2.shape
  D_local = D // axis_size

  half_D = D_local // 2
  W2_tiles = jnp.reshape(W2, (F_local, axis_size, 2, half_D)
      )

  cw_chunk = (axis_index + axis_size - 1) % axis_size
  ccw_chunk = (axis_index + 1) % axis_size

  acc_cw = Tmp @ W2_tiles[:, cw_chunk, 0, :]
  acc_ccw = Tmp @ W2_tiles[:, ccw_chunk, 1, :]

  def f(carry, step):
    acc_cw, acc_ccw = carry

    perm_cw = [(i, (i+1) % axis_size) for i in range(
        axis_size)]
    perm_ccw = [(i, (i-1) % axis_size) for i in range(
        axis_size)]

    next_cw = jax.lax.ppermute(acc_cw, 'y', perm=perm_cw)
    next_ccw = jax.lax.ppermute(acc_ccw, 'y', perm=perm_ccw)

    cw_idx = (axis_index + axis_size - 1 - step) % axis_size
    ccw_idx = (axis_index + 1 + step) % axis_size

    local_cw = Tmp @ W2_tiles[:, cw_idx,0, :]
    local_ccw = Tmp @ W2_tiles[:, ccw_idx,1, :]

    return (next_cw+local_cw, next_ccw+local_ccw), None

  steps = jnp.arange(1, axis_size)

  (final_cw, final_ccw), _ = jax.lax.scan(f, (acc_cw,
      acc_ccw), steps)

  final_acc = jnp.concatenate([final_cw, final_ccw], axis
      =-1)

  return final_acc
```

```
import time

B = 4096
D = 8192
F = 8192

W = jnp.zeros((F,D))
Tmp = jnp.zeros((B,F))

start_clock = time.perf_counter()

out = overlapped_reduce_scatter(Tmp,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")

start_clock = time.perf_counter()

out = bidirectional_reduce_scatter(Tmp,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms")
```

Note: Bidirectional reduce scatter is 1.23 times faster than unidirectional reduce scatter.

```
@jax.shard_map(
    mesh=mesh,
    in_specs=(jax.P('x','y'), jax.P('y', None)),
    out_specs=jax.P('x', None),
    check_vma=False
)
def bidirectional_allreduce(A, W):
  axis_size = jax.lax.axis_size('y')
  B_X, D_Y = A.shape
  _, F = W.shape

  partial = A @ W
  half_F = F // 2

  forward_accum = jax.lax.slice_in_dim(partial, start_index
      =0, limit_index=half_F, axis=1)
  backward_accum = jax.lax.slice_in_dim(partial, start_index
      =half_F, limit_index=F, axis=1)
```

```
  forward_send = forward_accum
  backward_send = backward_accum

  forward_perm = [(i, (i + 1) % axis_size) for i in range(
      axis_size)]
  backward_perm = [(i, (i - 1) % axis_size) for i in range(
      axis_size)]

  def f(i, carry):
    f_acc, b_acc, f_send, b_send = carry

    next_f_send = jax.lax.ppermute(f_send, axis_name='y',
        perm=forward_perm)
    next_b_send = jax.lax.ppermute(b_send, axis_name='y',
        perm=backward_perm)

    next_f_acc = f_acc + next_f_send
    next_b_acc = b_acc + next_b_send

    return (next_f_acc, next_b_acc, next_f_send, next_b_send
        )

  final_f_acc, final_b_acc, _, _ = jax.lax.fori_loop(
      0,
      axis_size - 1,
      f,
      (forward_accum, backward_accum, forward_send,
          backward_send)
  )

  return jax.lax.concatenate([final_f_acc, final_b_acc],
      dimension=1)

import time

B = 4096
D = 4096
F = 4096

W = jnp.zeros((D,F))
A = jnp.zeros((B,D))

start_clock = time.perf_counter()

out = overlapped_allreduce(A,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms") # outputs ~4672.5
```

```
    ms

start_clock = time.perf_counter()

out = bidirectional_allreduce(A,W)
out.block_until_ready()

end_clock = time.perf_counter()

print(f"{(end_clock-start_clock)*1000}ms") # outputs ~3713.5
    ms
```

Note: The bidirectional all reduce is 1.26 times faster than the unidirectional all reduce.

Part 11: Conclusions

No exercises in this chapter.

Part 12: GPUs

Quiz 1

1. H100 has $132 \cdot 4 \cdot 32 = 16896$ ALUs.
   B200 has $148 \cdot 4 \cdot 32 = 18944$ ALUs.
   TPUv5p has $2 \cdot 4 \cdot 8 \cdot 128 = 8192$ ALUs.

2. $16896 \cdot 1.59$ GHz $= 26.9$ TFLOPs.
   With boost, $16896 \cdot 1.98$ GHz $= 33.5$ TFLOPs.

   Matmul FLOPs are $9.9 \cdot 10^{14}$ per second, so you get around 29.6 times more FLOPs.

3.

$$\text{H100 peak fp16 intensity} = \frac{9.9 \cdot 10^{14}}{3.4 \cdot 10^{12}} = 291$$
$$\text{H100 peak fp8 intensity} = \frac{2.0 \cdot 10^{15}}{3.4 \cdot 10^{12}} = 588$$
$$\text{B200 peak fp16 intensity} = \frac{2.3 \cdot 10^{15}}{8.0 \cdot 10^{12}} = 288$$
$$\text{B200 peak fp8 intensity} = \frac{4.5 \cdot 10^{15}}{8.0 \cdot 10^{12}} = 563$$

4. Since $BS < 281$, $[64, 4096]@[4096, 8192]$ is comms bound.

$$2 \cdot 64 \cdot 4096 + 2 \cdot 4096 \cdot 8192 + 2 \cdot 64 \cdot 8192 = 68681728$$
$$\frac{68681728}{8.0 \cdot 10^{12}} = 8.6\mu\text{s}$$

$[512, 4096]@[4096, 8192]$ is compute bound since $512 > 281$.

$$\text{Total FLOPs} = 2 \cdot 512 \cdot 4096 \cdot 8192 = 34359738368$$
$$\frac{34359738368}{2.3 \cdot 10^{15}} = 14.9\mu\text{s}$$

5. SMEM capacity H100 = 256kb.
Each SM register also has 256kb capacity.

$$2 \cdot 132 \cdot 256\text{kb} = 66\text{MB combined register and SMEM capacity}$$

TPU VMEM is 120MB, but register memory is 256kb, equivalent to a single SM.

6. CUDA cores $\simeq$ ALUs.
$148 \cdot 4 \cdot 32 = 18944$ ALUs.
$18944 \cdot 2 = 37888$ FLOPs / cycle.

$$\frac{8.0 \cdot 10^{12}}{37888} = 2.11 \text{ GHz (cycles / s)}$$

7. Adding two fp32[N] vectors requires N FLOPs and $4 \cdot 2 \cdot N + 4N = 12N$ bytes.
Its a 1 / 12 ratio which is quite bad.
Assuming $N = 512$, this would take:

$$T_{\text{comms}} = \frac{12 \cdot 512}{3.4 \cdot 10^{12}} = 1.8 \text{ ns}$$

Quiz 2

1. NVSwitch bandwidth $= 4 \cdot 64 \cdot 25 \cdot 10^9 = 6.4$ TB/s
Each GPU only has 450GB in bandwidth because it is limited by the link level.

2. $8/2 = 4$, so 4 GPUs per half means $4 \cdot 450$ GB/s for each section. Bidirectional flow means $8 \cdot 450$ GB/s.

3. We need to make $N - 1$ sends, and send $B/(N \cdot W_{\text{unidirectional}})$ bytes.

$$(N-1) \cdot (B/(N \cdot W_{\text{unidirectional}}))$$
$$B = 2 \cdot 4096 \cdot 65356 = 512\text{MB}$$
$$7 \cdot (512\text{MB}/8 \cdot 450\text{GB}) = 1\text{ms}$$

Quiz 3

1. Node to Leaf $= 8 \cdot 400/8 = 400$ GB/s per node. This is egress.
Node to Leaf (ingress) $= 32 \cdot 400/8 = 12.8$ TB/s.
(12.8 TB/s)/32 = 400 GB/s per node.
Leaf to Spine (ingress) $= \frac{8 \cdot 16 \cdot 2 \cdot 400}{8} = 12.8$ TB/s.
Divided by 32, this is 400 GB/s per node.
Max spine capacity $= \frac{16 \cdot 64 \cdot 400}{8} = 51.2$ TB/s.
$\frac{51.2}{128} = 400$ GB/s per node BW always.

2. We could double the number of spine switches, assuming each new GPU is set in the SU structure. If we ran out of ports in the leaf switches we either need to switch to 1x400 cables for spine connections, or include more switch levels in the tree.

Pop Quizzes

Pop Quiz 1:

$$B = 2 \cdot 1024 \cdot 16384 = 34\text{MB}$$
$$34 \cdot 10^6/450 \cdot 10^9 = 76\mu\text{s}$$

Pop Quiz 2:

$$\frac{B}{WN} \quad W$$

$$\frac{(N-1)}{N} \cdot \frac{1}{2} \cdot \frac{B}{WN} = \frac{BN-B}{2WN^2}$$

Pop Quiz 3:

$$\text{If } Y \leq 8 : \frac{2DF \cdot 31}{32 \cdot 400 \cdot 10^9}$$
$$\text{If } Y > 8 : \frac{2DF \cdot 256}{Y \cdot 32 \cdot 400\text{GB}} = \frac{2DF}{Y \cdot 50 \cdot 10^9}$$

Quiz 4

1. Node to Leaf (ingress) = $\frac{B}{MN} \cdot N = \frac{B}{M}$
Egress to spine = $\frac{B}{M}$
Ingress from spine = $\frac{B \cdot (M-1)}{M}$
Egress to GPUs = $N \cdot (B - \frac{B}{MN}) = BN - \frac{B}{M}$
Ingress = $\frac{B+BM-B}{M} = \frac{BM}{M} = B$
Egress = $\frac{B+BNM-B}{M} = BN$
$T_{\text{AG}} = \frac{BN-B}{W} = \frac{B}{450 \cdot 10^9}$
Spine calculation:
$\frac{B}{M} \cdot M = B$ (Ingress)
$M \cdot (B \cdot \frac{M-1}{M})$ (Egress)
$B(M-1)$
$T_{\text{AG}} = \frac{B(M-1)}{M \cdot 400 \cdot 10^9}$

2. Ingress to SHARP switch = $N \cdot (B \cdot \frac{N-1}{N}) = B(N-1)$
Egress from SHARP switch = $\frac{B}{N} \cdot N = B$
Ingress to SHARP switch after psum = $N \cdot \frac{B}{N} = B$
Egress from SHARP switch = $N \cdot (B \cdot \frac{N-1}{N}) = B(N-1)$
Ingress = Egress = $BN/N = B$ bytes per cable
Time = $\frac{B}{W_{\text{egress}}}$

3. Egress from GPUs = $\frac{B(X-1)}{XY} \cdot N$
Ingress to GPUs = $\frac{B}{XY} \cdot N$
Egress from GPUs = $\frac{B}{XY} \cdot N$
Ingress to GPUs = $\frac{B(X-1)}{XY} \cdot N$
Ingress = Egress = $\frac{BN}{Y}$
$T_{\text{comms}} = \frac{N \cdot 2DF}{Y \cdot N \cdot W} = \frac{2DF}{YW}$

4. Lets assume we can use SHARP.

Total cluster bandwidth $= 32 \cdot 800 = 25.6$ TB/s
Divide by number of spine switches $= 6.4$ TB/s
$2DF/6.4$ TB/s
Note: This problem assumes a node is a spine switch.

5. T_{comms} at node $= \frac{B \cdot 7}{8 \cdot 450 \cdot 10^9} = \frac{13}{514 \cdot 10^9}$
T_{comms} in scale out network $= \frac{B}{2 \cdot 400 \cdot 10^9} = \frac{B}{800 \cdot 10^9}$
Bounded by comms at node.

Quiz 5

1. Within node DP roofline $= \frac{B}{X} > \frac{2.3 \cdot 10^{15}}{400 \cdot 10^9}$
So B can be twice as small as before.
Scale out network DP roofline $= \frac{B}{X} > \frac{2.3 \cdot 10^{15}}{400 \cdot 10^9}$
So in this case we would want to shard intra node.
Within node TP roofline $= Y < \frac{F \cdot 400 \cdot 10^9}{2.3 \cdot 10^{15}}$
So we can double the amount of nodes used.
Scale out network TP roofline $= Y < \frac{F \cdot 400 \cdot 10^9}{2.3 \cdot 10^{15}}$
In general, beyond a node it is hard to be compute bound.

2. $(4 + 4 + 2)(70\text{B}) = 700\text{GB}$
So we would need 9 H100s minimum.
$\frac{15\text{T}}{4096} = 3.6\text{B}$ per H100
Total FLOPs $= 6 \cdot 15\text{T} \cdot 70\text{B}$
Total compute $= 0.45 \cdot 4096 \cdot 9.9 \cdot 10^{14}$
$\frac{6 \cdot 15\text{T} \cdot 70\text{B}}{0.45 \cdot 4096 \cdot 9.9 \cdot 10^{14}} = 40$ days
$Y < \frac{28672 \cdot 450 \cdot 10^9}{9.9 \cdot 10^{14}} \leq 13$
So we can only use one node, but $8 \cdot 80 < 700$.
We can do 512 way Zero3 meaning our per GPU batch size is $4 \cdot 10^6/4096 = 976$. This is too low.
8 way pipelining means our AR/AG BW becomes $8 \cdot 400 = 3200$ GB/s.

$$990 \cdot 10^{12}/3200 \cdot 10^9 = 309, \text{ so we can use pipelining.}$$

3.

$$16\text{B} = (192 \cdot 4096)/192 = 4096$$
$$70\text{B} = (384 \cdot 4096)/768 = 2048$$
$$314\text{B} = (1536 \cdot 4096)/3072 = 2048$$

DP bound is 2500 so these are all either close to, or exceeding that.